Concept-to-Animation: LLM-Generated Manim Explainer Videos

CSE 434 Midterm Check-in Report

Jonah Femiani Koray Bayram

November 13, 2026

Team: Concept-to-Animation **Roles:** *Jonah* — baseline pipeline (LLM client, planner, code generation, render harness, logging, ablation runner), repair and layout loops, analysis, demo. *Koray* — few-shot curation, RAG corpus and retriever, narration. *Both* — human evaluation and reports.

I. Project Overview

Given a short description of a math or CS concept (e.g. "binary search"), our system writes a short explainer video in the style of 3Blue1Brown by having an LLM plan a storyboard and then write Manim Community Edition (ManimCE) code for each scene. LLM-written animation code fails in two characteristic ways: it calls *hallucinated or outdated Manim APIs* (the code crashes), and it produces *bad layouts* (objects overlap or leave the frame). Our research question is: **which generative techniques make LLM-written animation code render reliably and look clear?** We answer it with a cumulative ablation that starts from a zero-shot baseline and adds one technique at a time: error-driven self-repair, few-shot examples, retrieval-augmented generation (RAG) over the ManimCE docs, and a layout-repair loop driven by a bounding-box checker. This connects directly to course topics on prompting, in-context learning, RAG, iterative self-refinement, and LLM code generation.

II. Dataset

Concepts. We curated 6 concepts in three difficulty tiers of two each, chosen by how directly the idea maps to pictures: *concrete* (Pythagorean theorem, derivative as slope), *structural* (matrix multiplication, binary search), and *abstract* (recursion, Big-O notation). Each has a one-sentence prompt; the tiers let us ask whether performance drops as concepts get less visual.

Storyboards. For each concept the planner produces a storyboard of 3–4 scenes (20 scenes in total), each with a title, a visual description and narration. Storyboards are validated against a schema and cached, so every condition generates code for exactly the same scenes and only the code-generation step differs. All 6 were produced in one batch on 2026-11-10 by the same model.

Retrieval corpus (planned): the ManimCE docs and example gallery (MIT license), pinned to our render version (v0.22.1), chunked and embedded for RAG.

Ethics. The data contains no personal information. The main risk is that a fluent, polished video can teach *wrong* math; render success does not catch this, so we add human ratings of mathematical correctness. Generated videos will be labeled as AI-generated.

III. Methods

Figure 1 shows the current system (v1). A **planner** turns the concept into a storyboard. A **code generator** writes one ManimCE `Scene` subclass per storyboard scene; its prompt states the pinned Manim version, forbids the older ManimGL library, and gives the frame bounds ($x \in [-7.1, 7.1]$, $y \in [-4, 4]$). Post-processing is deliberately minimal (extract the code block, rename the class) so it cannot silently fix the errors we are measuring. Each scene is **rendered** in a separate subprocess with a 120 s timeout; we check that the expected video file was actually written, because Manim exits successfully even when the named scene does not exist. Failures are **classified** from the traceback into `hallucinated_api`, `bad_signature`, `latex`, `timeout`, `syntax` or `other`. Manim's `Mobject`

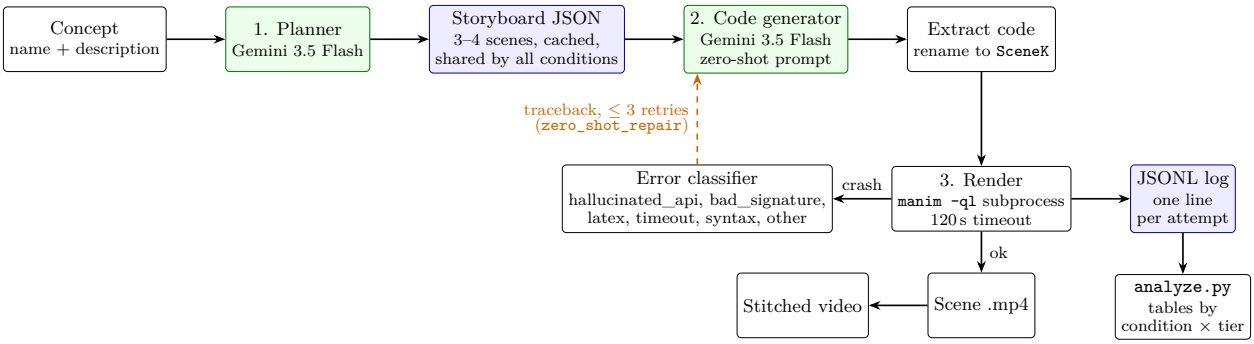


Figure 1: System design v1. Green: LLM calls; blue: stored artifacts. Solid path: the zero-shot baseline. Dashed: the error-repair loop (`zero_shot_repair`), which feeds the traceback back to the code generator. Few-shot, RAG and layout repair will attach to the code generator later.

invents any `get_*` method on demand, so a hallucinated call like `axes.get_secant_line(...)` surfaces as a `TypeError`; our classifier still labels it a hallucinated API. Every attempt is logged as one JSONL line (tokens, latency, error, code path), and responses are cached so reruns cost nothing.

Experimental design. The full study is a cumulative ablation over six conditions: `zero_shot`, `zero_shot_repair`, `few_shot`, `rag`, `rag_repair` and `rag_repair_layout`. Temperature is fixed at 0.2, conditions without repair get exactly one render attempt, and repair conditions get up to three extra attempts. Per-scene metrics, aggregated by condition and tier: render success, first-try success, repair iterations, layout-defect rate, error types, and two human scores (failed scene = 0, so they reflect the whole pipeline).

Changes since the proposal (approved by Dr. Zhang). (1) *Gemini 3.5 Flash instead of GPT + an open model:* we tried Qwen2.5-Coder (7B and 3B, 4-bit via Ollama), but its Manim code was too poor to make the comparison informative; open models stay available behind our LLM interface. (2) *No LangChain:* plain Python lets us log every prompt, reply and token count exactly. (3) *6 concepts instead of 40,* so every condition can be run and every video hand-rated.

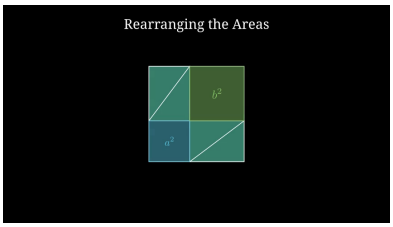
IV. Preliminary Results

Table 1 reports both conditions on all 20 scenes. Zero-shot renders 70% of scenes, roughly evenly across tiers; error-driven repair raises this to **100%**, needing 0.45 extra attempts per scene on average. Structural scenes (matrix grids, arrays) needed the most help (0.83 repairs per scene), so at this stage difficulty tracks how much layout code a scene needs, not how abstract the concept is. Repair costs about 2.3× the tokens per scene (4.3k vs. 1.9k). One caveat: every call reported its model as `gemini-3.6-flash`, so `gemini-3.5-flash` is an alias; we log the version to detect drift.

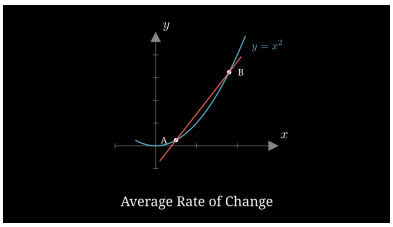
Qualitative examples. *Success:* all four Pythagorean-theorem scenes rendered first try; scene 3 (Fig. 2a) rearranges four triangles in an $(a + b)$ square to expose a^2 and b^2, a clean, correct version of the classic proof. *Contrasting failure:* for the derivative, scene 1 (Fig. 2b) correctly draws a secant through two points on $y = x^2$, but scenes 2 and 3, which must *animate* the secant approaching the tangent, both crashed zero-shot: the model called convenience methods (`axes.get_secant_line`, `get_tangent_line`) that sound right but do not exist in ManimCE. Given the traceback, repair replaced each with a small helper method that computes the line itself, and both scenes rendered on the second attempt. Repair is not always one step: in matrix multiplication scene 2, fixing an invalid keyword exposed an undefined color `CYAN` (which the model then defined itself), then a LaTeX error; it rendered on the fourth and last attempt.

	Automatic			Human (0–5; failed scene = 0)	
Condition	Render success	First-try success	Avg. repairs	Correctness	Clarity
`zero_shot`	70% (14/20)	70%	–	?	?
concrete (7)	71% (5/7)	71%	–	?	?
structural (6)	67% (4/6)	67%	–	?	?
abstract (7)	71% (5/7)	71%	–	?	?
`zero_shot_repair`	**100% (20/20)**	70%	0.45	?	?
concrete (7)	100% (7/7)	71%	0.29	?	?
structural (6)	100% (6/6)	67%	0.83	?	?
abstract (7)	100% (7/7)	71%	0.29	?	?

Table 1: Gemini 3.5 Flash, all 20 scenes (scenes per tier in parentheses). For `zero_shot_repair`, render minus first-try success is what repair adds; avg. repairs is over scenes that eventually rendered. Human scores: mean of both raters over all 20 scenes; rendered scenes rated 1–5 blind to condition, failed scenes scored 0.



(a) Pythagorean theorem, scene 3 (rendered).



(b) Derivative as slope, scene 1 (rendered).

Figure 2: Final frames from two zero-shot scenes that rendered on the first try.

Error types. All six zero-shot failures are API misuse, never syntax: three `bad_signature` (e.g. `grid_stroke_width`), two `hallucinated_api`, and one misused `TransformMatchingTex`. Each error names the bad call, which is why repair works so well and what RAG over the real docs should prevent up front.

Human ratings. [TODO: mean correctness and clarity per condition (failed scenes = 0), inter-rater agreement, and what the ratings show that render success does not.]

V. Next Steps

- **Few-shot and RAG** (Koray): 3–5 curated modern ManimCE example scenes, and a retriever over the pinned ManimCE docs checked on whether it returns the right classes (e.g. `Axes.plot` for graphs).
- **Layout repair:** snapshot bounding boxes after each animation, flag off-frame objects and unintended overlaps, and describe them to the LLM in plain language. Since repair already reaches 100% render success, clarity and correctness are now the metrics that can separate the remaining conditions.
- **Evaluation and demo:** the full six-condition ablation by tier, cost per condition, a success/failure gallery, gTTS narration and a type-a-concept web demo.

Open questions for feedback. Are 20 scenes enough to separate conditions? Is a bounding-box check a fair proxy for clarity, or should clarity rest on human ratings alone? Should we add leave-one-out rows to check the ablation's stacking order?